fn make_quantile_score_candidates

Michael Shoemate Christian Covington Ira Globus-Harrus

March 10, 2023

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `make_quantile_score_candidates` in `mod.rs` at commit f5bb719 (outdated[1]). `make_quantile_score_candidates` returns a Transformation that takes a numeric vector database and a vector of numeric quantile candidates, and returns a vector of scores, where higher scores correspond to more accurate candidates.

Vetting History

- Pull Request #456

1 Intuition

The quantile score function scores each c in a set of candidates C.

$$s_i \quad = -|(1-\alpha)\#(X < C_i) - \alpha\#(X > C_i)| \tag{1}$$

Where $\#(X < C_i) = |\{x \in X | x < C_i\}|$ is the number of values in X less than C_i, and similarly for other variations of inequalities. The scalar score function can be equivalently stated:

$$\begin{aligned} s_i &= -|(1-\alpha)\#(X < c) - \alpha\#(X > c)| && (2)\\ &= -|(1-\alpha)\#(X < c) - \alpha * (|X| - \#(X < c) - \#(X = c))| && (3)\\ &= -|\#(X < c) - \alpha * (|X| - \#(X = c))| && (4) \end{aligned}$$

It has an intuitive interpretation as $-|candidate_rank - ideal_rank|$, where the absolute distance between the candidate and ideal rank penalizes the score. The ideal rank does not include values in the dataset equal to the candidate. The score is maximized at zero when the candidate rank is equivalent to the rank at the ideal α-quantile.

The scalar scorer is almost equivalent to Smith's[1], but adjusts for a source of bias when there are values in the dataset equal to the candidate. For comparison, we can equivalently write the OpenDP scorer as if there were some α-discount on dataset entries equal to the candidate.

$$\begin{array}{cl} OpenDP & -|\#(X < c) + \alpha\#(X = c) - \alpha * |X|| \\ Smith & -|\#(X < c) + 1\#(X = c) - \alpha * |X|| \end{array}$$

Observing that $\#(X \leq c) = \#(X < c) + 1\#(X = c)$.

[1] See new changes with `git diff f5bb719..093922f9 rust/src/transformations/quantile_score_candidates/mod.rs`

1.1 Examples

Let $X = \{0, 1, 2, 3, 4\}$ and $\alpha = 0.5$ (median):

$$
\begin{aligned}
score(X, 0, \alpha) &= -|0 - .5 * (5 - 1)| = -2 \\
score(X, 1, \alpha) &= -|1 - .5 * (5 - 1)| = -1 \\
score(X, 2, \alpha) &= -|2 - .5 * (5 - 1)| = -0 \\
score(X, 3, \alpha) &= -|3 - .5 * (5 - 1)| = -1 \\
score(X, 4, \alpha) &= -|4 - .5 * (5 - 1)| = -2
\end{aligned}
$$

The score is maximized by the candidate at the true median.
Let $X = \{0, 1, 2, 3, 4, 5\}$ and $\alpha = 0.5$ (median):

$$
\begin{aligned}
score(X, 0, \alpha) &= -|0 - .5 * (6 - 1)| = -2.5 \\
score(X, 1, \alpha) &= -|1 - .5 * (6 - 1)| = -1.5 \\
score(X, 2, \alpha) &= -|2 - .5 * (6 - 1)| = -0.5 \\
score(X, 3, \alpha) &= -|3 - .5 * (6 - 1)| = -0.5 \\
score(X, 4, \alpha) &= -|4 - .5 * (6 - 1)| = -1.5 \\
score(X, 5, \alpha) &= -|5 - .5 * (6 - 1)| = -2.5
\end{aligned}
$$

The two candidates nearest the median are scored equally and highest.
Let $X = \{0, 1, 2, 3, 4\}$ and $\alpha = 0.25$ (first quartile):

$$
\begin{aligned}
score(X, 0, \alpha) &= -|0 - .25 * (5 - 1)| = -1 \\
score(X, 1, \alpha) &= -|1 - .25 * (5 - 1)| = -0 \\
score(X, 2, \alpha) &= -|2 - .25 * (5 - 1)| = -1 \\
score(X, 3, \alpha) &= -|3 - .25 * (5 - 1)| = -2 \\
score(X, 4, \alpha) &= -|4 - .25 * (5 - 1)| = -3
\end{aligned}
$$

As expected, the score is maximized when $c = 1$.
Let $X = \{0, 1, 2, 3, 4, 5\}$ and $\alpha = 0.25$ (first quartile):

$$
\begin{aligned}
score(X, 0, \alpha) &= -|0 - .25 * (6 - 1)| = -1.25 \\
score(X, 1, \alpha) &= -|1 - .25 * (6 - 1)| = -0.25 \\
score(X, 2, \alpha) &= -|2 - .25 * (6 - 1)| = -0.75 \\
score(X, 3, \alpha) &= -|3 - .25 * (6 - 1)| = -1.75 \\
score(X, 4, \alpha) &= -|4 - .25 * (6 - 1)| = -2.75 \\
score(X, 5, \alpha) &= -|5 - .25 * (6 - 1)| = -3.75
\end{aligned}
$$

The ideal rank is 1.25. The nearest candidate, 1, has the greatest score, followed by 2, and then 0.

2 Hoare Triple

Precondition

- `TIA` (input atom type) is a type with trait `Number`.
- `TOA` (output atom type) is a type with trait `Float`.

Function

```python
def make_quantile_score_candidates(candidates: List[TIA], alpha: TOA):

    for i in range(len(candidates) - 1):
        assert candidates[i] < candidates[i + 1]

    assert 0 <= alpha <= 1

    abs_dist_const = max(alpha, (1).inf_sub(alpha))
    sup_dist_const = abs_dist_const.inf_mul(2)

    def function(arg: List[TIA]):
        return score(arg, candidates, alpha)

    def stability_map(d_in: u32):
        return TOA.inf_cast(d_in).inf_mul(sup_dist_const)

    return Transformation(
        input_domain=VectorDomain(AllDomain(TIA)),
        output_domain=VectorDomain(AllDomain(TOA)),
        function=function,
        input_metric=SymmetricDistance(),
        output_metric=InfDifferenceDistance(TOA),
        stability_map=stability_map,
    )
```

Postcondition

For every setting of the input parameters (`candidates`, `alpha`) to `make_quantile_score_candidates` such that the given preconditions hold, `make_quantile_score_candidates` raises an exception (at compile time or run time) or returns a valid transformation. A valid transformation has the following properties:

1. (Appropriate output domain). For every element v in `input_domain`, `function`(v) is in `output_domain` or raises a data-independent runtime exception.

2. (Domain-metric compatibility). The domain `input_domain` matches one of the possible domains listed in the definition of `input_metric`, and likewise `output_domain` matches one of the possible domains listed in the definition of `output_metric`.

3. (Stability guarantee). For every pair of elements u, v in `input_domain` and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for `output_metric`, if u, v are `d_in`-close under `input_metric` and `stability_map`(`d_in`) $\leq$ `d_out`, then `function`(u), `function`(v) are `d_out`-close under `output_metric`.

3 Proof

3.1 Appropriate Output Domain

The raw type and domain are equivalent, save for potential nullity in the atomic type. The scalar scorer structurally cannot emit null, because the input argument is non-null.

3.2 Domain-metric compatibility

On the input side, SymmetricDistance is well-defined on VectorDomains. On the output side, InfDifferenceDistance is well-defined on VectorDomains.

3.3 Stability Guarantee

Lemma 3.1. If $d_{Sym}(X, X') = 1$, then $d_\infty(\texttt{function}(X), \texttt{function}(X')) \leq max(\alpha, 1-\alpha)$.

Proof. Assume $d_{Sym}(X, X') = 1$. For convenience, let $s = \texttt{function}(X)$. Consider each of the three cases of adding or removing an element in X.

$$d_\infty(s_i, s_i') = \max_i |s_i - s_i'| \qquad \text{by definition of } d_\infty$$

Case 1. Assume X' is equal to X, but with some $X_j < C_i$ added or removed.

$$\begin{aligned}
&= \max_{s \sim s'} \max_i ||(1-\alpha)\#(X < C_i) - \alpha\#(X > C_i)| \\
&\qquad - |(1-\alpha)(\#(X < C_i) \pm 1) - \alpha\#(X > C_i)|| \\
&\leq \max_{s \sim s'} \max_i ||(1-\alpha)\#(X < C_i) - \alpha\#(X > C_i)| \\
&\qquad - (|(1-\alpha)\#(X < C_i) - \alpha\#(X > C_i)| + |\pm(1-\alpha)|)| && \text{by triangle inequality} \\
&= \max_{s \sim s'} \max_i |1-\alpha| && \text{scores cancel} \\
&= 1-\alpha && \text{since } \alpha \leq 1
\end{aligned}$$

Case 2. Assume X' is equal to X, but with some $X_j > C_i$ added or removed.

$$\begin{aligned}
&= \max_{s \sim s'} \max_i ||(1-\alpha)\#(X < C_i) - \alpha\#(X > C_i)| \\
&\qquad - |(1-\alpha)\#(X < C_i) - \alpha(\#(X > C_i) \pm 1)|| \\
&\leq \max_{s \sim s'} \max_i ||(1-\alpha)\#(X < C_i) - \alpha\#(X > C_i)| \\
&\qquad - (|(1-\alpha)\#(X < C_i) - \alpha\#(X > C_i)| + |\pm\alpha|)| && \text{by triangle inequality} \\
&= \max_{s \sim s'} \max_i |\alpha| && \text{scores cancel} \\
&= \alpha && \text{since } \alpha \geq 0
\end{aligned}$$

Case 3. Assume X' is equal to X, but with some $X_j = C_i$ added or removed.

$$\begin{aligned}
&= \max_{s \sim s'} \max_i ||(1-\alpha)\#(X < C_i) - \alpha\#(X > C_i)| \\
&\qquad - |(1-\alpha)\#(X < C_i) - \alpha\#(X > C_i)|| \\
&= 0 && \text{no change in score}
\end{aligned}$$

Take the union bound over all cases.

$$d_\infty(s_i, s_i') = max(\alpha, 1-\alpha)$$

□

Take any two elements X, X' in the `input_domain` and any pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for `output_metric`. Assume X, X' are `d_in`-close under `input_metric` and that `stability_map`(`d_in`) $\leq$ `d_out`.

$$
\begin{aligned}
\texttt{d_out} &= \max_{X \sim X'} d_{IDD}(s, s') && \text{where } s = \texttt{function}(X) \\
&= \max_{X \sim X'} \max_{ij} |(s_i - s'_i) - (s_j - s'_j)| && \text{by definition of } \texttt{InfDifferenceDistance} \\
&\leq 2 \max_{X \sim X'} \max_{i} |s_i - s'_i| && s \text{ is not monotonic; take looser bound} \\
&\leq 2 \sum_{j}^{\texttt{d_in}} \max_{Z_j \sim Z_{j+1}} \max_{i} |s_{i,j} - s_{i,j+1}| && \text{by path property where } d_{Sym}(Z_i, Z_{i+1}) = 1, X = Z_0 \text{ and } Z_{\texttt{d_in}} = X' \\
&\leq 2 \sum_{j}^{\texttt{d_in}} max(\alpha, 1 - \alpha) && \text{by 3.1} \\
&\leq 2 \cdot \texttt{d_in}
\end{aligned}
$$

It is shown that `function`(X), `function`(X') are `d_out`-close under `output_metric`.

References

[1] Adam Smith. Privacy-preserving statistical estimation with optimal convergence rates. In *Proceedings of the Forty-Third Annual ACM Symposium on Theory of Computing*, STOC '11, page 813–822, New York, NY, USA, 2011. Association for Computing Machinery.